

1 Install

```
!pip install tensorflow scikit-learn nltk --quiet
```

2 Imports

```
import pandas as pd
import numpy as np
import tensorflow as tf
import nltk
nltk.download('stopwords')

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, GRU, SimpleRNN, Dense, Dropout, Bidirectional
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
```

3 Upload Dataset

```
df = pd.read_csv("/content/Fake_Real_News_Data.csv")

df = df[['title', 'text', 'label']]
df.dropna(inplace=True)

# combine
df['content'] = df['title'] + " " + df['text']
df = df[['content', 'label']]

print("Total samples:", len(df))
df.head()
```

Total samples: 6335

	content	label
0	A whirlwind day in D.C. showcases Trump's unor...	REAL
1	In Baltimore's call for federal police probe, ...	REAL
2	Trump Proudly Declares: Most Of The People I'v...	FAKE
3	Inside the Trump-Bush melodrama: Decades of te...	REAL
4	Shutdown clash to return in force by December ...	REAL

4 Split

```
X = df['content']
y = df['label']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

5 Tokenization

```
VOCAB_SIZE = 50000
MAX_LEN = 400

tokenizer = Tokenizer(num_words=VOCAB_SIZE, oov_token="<OOV>")
tokenizer.fit_on_texts(X_train)

train_seq = tokenizer.texts_to_sequences(X_train)
test_seq = tokenizer.texts_to_sequences(X_test)
```

6 PRE & POST Padding

```
# PRE
X_train_pre = pad_sequences(train_seq, maxlen=MAX_LEN, padding='pre')
X_test_pre = pad_sequences(test_seq, maxlen=MAX_LEN, padding='pre')

# POST
X_train_post = pad_sequences(train_seq, maxlen=MAX_LEN, padding='post')
X_test_post = pad_sequences(test_seq, maxlen=MAX_LEN, padding='post')
```

7 Model Builder

```
def build_model(model_type):

    model = Sequential()
    model.add(Embedding(VOCAB_SIZE, 128, input_length=MAX_LEN))
```

```

if model_type == "LSTM":
    model.add(Bidirectional(LSTM(64)))

elif model_type == "GRU":
    model.add(Bidirectional(GRU(64)))

elif model_type == "RNN":
    model.add(Bidirectional(SimpleRNN(64)))

model.add(Dropout(0.5))
model.add(Dense(1, activation='sigmoid'))

model.compile(
    loss='binary_crossentropy',
    optimizer='adam',
    metrics=['accuracy'])

return model

```

8 Evaluation Function

```

def evaluate_model(model, Xtest, ytest):

    prob = model.predict(Xtest, verbose=0)
    pred = (prob > 0.5).astype(int)

    acc = accuracy_score(ytest, pred)
    prec = precision_score(ytest, pred)
    rec = recall_score(ytest, pred)
    f1 = f1_score(ytest, pred)

    return acc, prec, rec, f1, prob

```

9 Fusion Function (PRE+POST)

```

def fusion_scores(prob1, prob2, ytrue):

    final = (prob1 + prob2) / 2
    final = (final > 0.5).astype(int)

    acc = accuracy_score(ytrue, final)
    prec = precision_score(ytrue, final)
    rec = recall_score(ytrue, final)
    f1 = f1_score(ytrue, final)

    return acc, prec, rec, f1

```

10 TRAIN ALL 9 MODELS (IMPORTANT CELL)

```

EPOCHS = 3
BATCH = 64

all_results = {}

# Convert y_train and y_test labels from strings to numerical (0 or 1)
y_train_encoded = y_train.apply(lambda x: 1 if x == 'REAL' else 0)
y_test_encoded = y_test.apply(lambda x: 1 if x == 'REAL' else 0)

for model_name in ["LSTM", "GRU", "RNN"]:

    print("\n\n=====")
    print("TRAINING:", model_name)
    print("=====")

    # PRE MODEL
    print("\n---", model_name, "PRE ---")
    pre_model = build_model(model_name)
    pre_model.fit(X_train_pre, y_train_encoded, epochs=EPOCHS,
                  batch_size=BATCH, validation_split=0.1, verbose=1)

    pre_acc, pre_prec, pre_rec, pre_f1, pre_prob = evaluate_model(pre_model, X_test_pre, y_test_encoded)
    all_results[model_name+"_PRE"] = [pre_acc, pre_prec, pre_rec, pre_f1]
    print("RESULT:", all_results[model_name+"_PRE"])

    # POST MODEL
    print("\n---", model_name, "POST ---")
    post_model = build_model(model_name)
    post_model.fit(X_train_post, y_train_encoded, epochs=EPOCHS,
                   batch_size=BATCH, validation_split=0.1, verbose=1)

    post_acc, post_prec, post_rec, post_f1, post_prob = evaluate_model(post_model, X_test_post, y_test_encoded)
    all_results[model_name+"_POST"] = [post_acc, post_prec, post_rec, post_f1]
    print("RESULT:", all_results[model_name+"_POST"])

    # FUSION MODEL
    print("\n---", model_name, "PRE+POST FUSION ---")
    fus = fusion_scores(pre_prob, post_prob, y_test_encoded)
    all_results[model_name+"_PRE+POST_FUSION"] = fus
    print("RESULT:", fus)

=====
TRAINING: LSTM
=====

--- LSTM PRE ---
Epoch 1/3
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:97: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
72/72 ----- 72s 936ms/step - accuracy: 0.6200 - loss: 0.6313 - val_accuracy: 0.7830 - val_loss: 0.4411
Epoch 2/3
72/72 ----- 64s 886ms/step - accuracy: 0.8662 - loss: 0.3215 - val_accuracy: 0.8797 - val_loss: 0.2950

```

```

Epoch 3/3
72/72 ————— 66s 915ms/step - accuracy: 0.9684 - loss: 0.1158 - val_accuracy: 0.8777 - val_loss: 0.3096
RESULT: [0.8871349644830308, 0.8736681887366818, 0.9053627760252366, 0.8892331525948877]

--- LSTM POST ---
Epoch 1/3
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:97: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
72/72 ————— 68s 892ms/step - accuracy: 0.6439 - loss: 0.6421 - val_accuracy: 0.8245 - val_loss: 0.3982
Epoch 2/3
72/72 ————— 64s 888ms/step - accuracy: 0.9098 - loss: 0.2433 - val_accuracy: 0.8738 - val_loss: 0.2668
Epoch 3/3
72/72 ————— 65s 905ms/step - accuracy: 0.9791 - loss: 0.0745 - val_accuracy: 0.8915 - val_loss: 0.2582
RESULT: [0.9060773480662984, 0.8955453149001537, 0.919558359621451, 0.9073929961089494]

--- LSTM PRE+POST FUSION ---
RESULT: (0.9250197316495659, 0.9152542372881356, 0.9369085173501577, 0.9259547934528449)

=====
TRAINING: GRU
=====

--- GRU PRE ---
Epoch 1/3
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:97: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
72/72 ————— 76s 998ms/step - accuracy: 0.6453 - loss: 0.6353 - val_accuracy: 0.7377 - val_loss: 0.4920
Epoch 2/3
72/72 ————— 69s 961ms/step - accuracy: 0.8930 - loss: 0.2871 - val_accuracy: 0.8501 - val_loss: 0.3431
Epoch 3/3
72/72 ————— 70s 974ms/step - accuracy: 0.9743 - loss: 0.0851 - val_accuracy: 0.8363 - val_loss: 0.4268
RESULT: [0.8571428571428571, 0.9081081081081082, 0.7949526813880127, 0.847771236333053]

--- GRU POST ---
Epoch 1/3
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:97: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
72/72 ————— 75s 974ms/step - accuracy: 0.6003 - loss: 0.6544 - val_accuracy: 0.7278 - val_loss: 0.5481
Epoch 2/3
72/72 ————— 71s 981ms/step - accuracy: 0.8578 - loss: 0.3509 - val_accuracy: 0.8402 - val_loss: 0.3774
Epoch 3/3
72/72 ————— 69s 953ms/step - accuracy: 0.8663 - loss: 0.3156 - val_accuracy: 0.7298 - val_loss: 0.5512
RESULT: [0.7348066298342542, 0.7098591549295775, 0.7949526813880127, 0.75]

```

10 Display Final Table

```

print("\n\nFINAL RESULTS")
for k,v in all_results.items():
    print(k,":",v)

```

```

FINAL RESULTS
LSTM_PRE : [0.8871349644830308, 0.8736681887366818, 0.9053627760252366, 0.8892331525948877]
LSTM_POST : [0.9060773480662984, 0.8955453149001537, 0.919558359621451, 0.9073929961089494]
LSTM_PRE+POST_FUSION : (0.9250197316495659, 0.9152542372881356, 0.9369085173501577, 0.9259547934528449)
GRU_PRE : [0.8571428571428571, 0.9081081081081082, 0.7949526813880127, 0.847771236333053]

```

```
GRU_POST : [0.7348066298342542, 0.7098591549295775, 0.7949526813880127, 0.75]
GRU_PRE+POST_FUSION : (0.8602999210734017, 0.8919382504288165, 0.8201892744479495, 0.8545603944124898)
RNN_PRE : [0.7205998421468035, 0.7380952380952381, 0.6845425867507886, 0.7103109656301145]
RNN_POST : [0.7008681925808997, 0.6888888888888889, 0.7334384858044164, 0.7104660045836516]
RNN_PRE+POST_FUSION : (0.7490134175217048, 0.7438271604938271, 0.7602523659305994, 0.7519500780031201)
```

```
import pandas as pd

data = [
["LSTM_PRE",0.8871349644830308,0.8736681887366818,0.9053627760252366,0.8892331525948877],
["LSTM_POST",0.9060773480662984,0.8955453149001537,0.919558359621451,0.9073929961089494],
["LSTM_PRE+POST_FUSION",0.9250197316495659,0.9152542372881356,0.9369085173501577,0.9259547934528449],

["GRU_PRE",0.8571428571428571,0.9081081081081082,0.7949526813880127,0.847771236333053],
["GRU_POST",0.7348066298342542,0.7098591549295775,0.7949526813880127,0.75],
["GRU_PRE+POST_FUSION",0.8602999210734017,0.8919382504288165,0.8201892744479495,0.8545603944124898],

["RNN_PRE",0.7205998421468035,0.7380952380952381,0.6845425867507886,0.7103109656301145],
["RNN_POST",0.7008681925808997,0.6888888888888889,0.7334384858044164,0.7104660045836516],
["RNN_PRE+POST_FUSION",0.7490134175217048,0.7438271604938271,0.7602523659305994,0.7519500780031201],
]

df = pd.DataFrame(data,columns=["Model","Accuracy","Precision","Recall","F1-Score"])

# round values
df.iloc[:,1:] = df.iloc[:,1:].round(3)

df
```

	Model	Accuracy	Precision	Recall	F1-Score
0	LSTM_PRE	0.887	0.874	0.905	0.889
1	LSTM_POST	0.906	0.896	0.920	0.907
2	LSTM_PRE+POST_FUSION	0.925	0.915	0.937	0.926
3	GRU_PRE	0.857	0.908	0.795	0.848
4	GRU_POST	0.735	0.710	0.795	0.750
5	GRU_PRE+POST_FUSION	0.860	0.892	0.820	0.855
6	RNN_PRE	0.721	0.738	0.685	0.710
7	RNN_POST	0.701	0.689	0.733	0.710
8	RNN_PRE+POST_FUSION	0.749	0.744	0.760	0.752

DATASET 2

🔴 CELL 1 – Import Libraries

```
# =====
# CELL 1: Import Libraries
```

```
# =====

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import re
import tensorflow as tf

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Embedding, LSTM, GRU, SimpleRNN
```

🚀 CELL 2 – Load Dataset

```
# =====
# CELL 2: Load Dataset
# =====

fake_df = pd.read_csv('/content/Fake.csv')
true_df = pd.read_csv('/content/True.csv')

fake_df['label'] = 0
true_df['label'] = 1

df = pd.concat([fake_df, true_df], axis=0)
df = df.sample(frac=1, random_state=42).reset_index(drop=True)

df['content'] = df['title'] + " " + df['text']

df = df[['content', 'label']]

print("Dataset shape:", df.shape)
df.head()
```

Dataset shape: (44898, 2)

	content	label
0	Ben Stein Calls Out 9th Circuit Court: Committ...	0
1	Trump drops Steve Bannon from National Securit...	1
2	Puerto Rico expects U.S. to lift Jones Act shi...	1
3	OOPS: Trump Just Accidentally Confirmed He Le...	0
4	Donald Trump heads for Scotland to reopen a go...	1

🚀 CELL 3 – Text Preprocessing

```
# =====
# CELL 3: Basic Cleaning
# =====

def clean_text(text):
    text = text.lower()
    text = re.sub(r'^a-zA-Z ', '', text)
    return text

df['content'] = df['content'].apply(clean_text)
```

🚀 CELL 4 – Tokenization

```
# =====
# CELL 4: Tokenization
# =====

max_words = 10000
max_len = 300

tokenizer = Tokenizer(num_words=max_words)
tokenizer.fit_on_texts(df['content'])

X = tokenizer.texts_to_sequences(df['content'])
y = df['label']

X_pre = pad_sequences(X, maxlen=max_len, padding='pre')
X_post = pad_sequences(X, maxlen=max_len, padding='post')

X_train_pre, X_test_pre, y_train, y_test = train_test_split(X_pre, y, test_size=0.2, random_state=42)
X_train_post, X_test_post, _, _ = train_test_split(X_post, y, test_size=0.2, random_state=42)
```

🚀 CELL 5 – Model Builder Function

```
# =====
# CELL 5: Model Builder
# =====

def build_model(model_type):
    model = Sequential()
    model.add(Embedding(max_words, 128, input_length=max_len))

    if model_type == 'LSTM':
        model.add(LSTM(64))
    elif model_type == 'GRU':
        model.add(GRU(64))
    elif model_type == 'RNN':
        model.add(SimpleRNN(64))

    model.add(Dense(1, activation='sigmoid'))
```

```

model.compile(loss='binary_crossentropy',
              optimizer='adam',
              metrics=['accuracy'])

return model

```

🔥 CELL 6 – Training Function

```

# =====
# CELL 6: Train and Evaluate
# =====

def train_and_evaluate(model_type, X_train, X_test, name):
    model = build_model(model_type)

    model.fit(X_train, y_train,
              epochs=10,
              batch_size=128,
              validation_split=0.1,
              verbose=1)

    preds = (model.predict(X_test) > 0.5).astype("int32")

    acc = accuracy_score(y_test, preds)
    prec = precision_score(y_test, preds)
    rec = recall_score(y_test, preds)
    f1 = f1_score(y_test, preds)

    return [name, acc, prec, rec, f1]

```

🔥 CELL 7 – Train All Models (PRE, POST, FUSION)

```

# =====
# CELL 7: Train All Models
# =====

results = []

# LSTM
results.append(train_and_evaluate('LSTM', X_train_pre, X_test_pre, "LSTM_PRE"))
results.append(train_and_evaluate('LSTM', X_train_post, X_test_post, "LSTM_POST"))

# GRU
results.append(train_and_evaluate('GRU', X_train_pre, X_test_pre, "GRU_PRE"))
results.append(train_and_evaluate('GRU', X_train_post, X_test_post, "GRU_POST"))

# RNN
results.append(train_and_evaluate('RNN', X_train_pre, X_test_pre, "RNN_PRE"))
results.append(train_and_evaluate('RNN', X_train_post, X_test_post, "RNN_POST"))

```

```

Epoch 1/10
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:97: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
253/253 ————— 6s 18ms/step - accuracy: 0.8714 - loss: 0.3082 - val_accuracy: 0.9190 - val_loss: 0.2174
Epoch 2/10
253/253 ————— 5s 18ms/step - accuracy: 0.9622 - loss: 0.1221 - val_accuracy: 0.9613 - val_loss: 0.1083
Epoch 3/10
253/253 ————— 4s 17ms/step - accuracy: 0.9742 - loss: 0.0863 - val_accuracy: 0.9727 - val_loss: 0.0873
Epoch 4/10
253/253 ————— 4s 16ms/step - accuracy: 0.9875 - loss: 0.0437 - val_accuracy: 0.9836 - val_loss: 0.0597
Epoch 5/10
253/253 ————— 5s 19ms/step - accuracy: 0.9910 - loss: 0.0327 - val_accuracy: 0.9805 - val_loss: 0.0704
Epoch 6/10
253/253 ————— 4s 16ms/step - accuracy: 0.9692 - loss: 0.0862 - val_accuracy: 0.9825 - val_loss: 0.0639
Epoch 7/10
253/253 ————— 4s 16ms/step - accuracy: 0.9915 - loss: 0.0327 - val_accuracy: 0.9699 - val_loss: 0.0971
Epoch 8/10
253/253 ————— 5s 18ms/step - accuracy: 0.9920 - loss: 0.0285 - val_accuracy: 0.9852 - val_loss: 0.0553
Epoch 9/10
253/253 ————— 4s 16ms/step - accuracy: 0.9979 - loss: 0.0094 - val_accuracy: 0.9830 - val_loss: 0.0647
Epoch 10/10
253/253 ————— 4s 17ms/step - accuracy: 0.9980 - loss: 0.0099 - val_accuracy: 0.9836 - val_loss: 0.0566
281/281 ————— 2s 7ms/step
Epoch 1/10
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:97: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
253/253 ————— 6s 17ms/step - accuracy: 0.7763 - loss: 0.5806 - val_accuracy: 0.7790 - val_loss: 0.3984
Epoch 2/10
253/253 ————— 4s 17ms/step - accuracy: 0.7764 - loss: 0.3715 - val_accuracy: 0.8082 - val_loss: 0.3331
Epoch 3/10
253/253 ————— 4s 18ms/step - accuracy: 0.8496 - loss: 0.2818 - val_accuracy: 0.9638 - val_loss: 0.1240
Epoch 4/10
253/253 ————— 4s 16ms/step - accuracy: 0.9786 - loss: 0.0805 - val_accuracy: 0.9719 - val_loss: 0.1021
Epoch 5/10
253/253 ————— 5s 18ms/step - accuracy: 0.9818 - loss: 0.0645 - val_accuracy: 0.9688 - val_loss: 0.0944
Epoch 6/10
253/253 ————— 4s 16ms/step - accuracy: 0.9869 - loss: 0.0448 - val_accuracy: 0.9758 - val_loss: 0.0862
Epoch 7/10
253/253 ————— 5s 20ms/step - accuracy: 0.9942 - loss: 0.0260 - val_accuracy: 0.9688 - val_loss: 0.0954
Epoch 8/10
253/253 ————— 5s 19ms/step - accuracy: 0.9951 - loss: 0.0203 - val_accuracy: 0.9808 - val_loss: 0.0750
Epoch 9/10
253/253 ————— 4s 16ms/step - accuracy: 0.9967 - loss: 0.0135 - val_accuracy: 0.9766 - val_loss: 0.0843
Epoch 10/10
253/253 ————— 4s 16ms/step - accuracy: 0.9962 - loss: 0.0183 - val_accuracy: 0.9833 - val_loss: 0.0683
281/281 ————— 2s 7ms/step
Epoch 1/10
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:97: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
253/253 ————— 5s 16ms/step - accuracy: 0.8664 - loss: 0.2994 - val_accuracy: 0.9749 - val_loss: 0.0771
Epoch 2/10
253/253 ————— 4s 16ms/step - accuracy: 0.9825 - loss: 0.0586 - val_accuracy: 0.9705 - val_loss: 0.0800
Epoch 3/10
253/253 ————— 4s 18ms/step - accuracy: 0.9872 - loss: 0.0426 - val_accuracy: 0.9861 - val_loss: 0.0494
Epoch 4/10
253/253 ————— 4s 16ms/step - accuracy: 0.9835 - loss: 0.0458 - val_accuracy: 0.9841 - val_loss: 0.0496
Epoch 5/10
253/253 ————— 4s 16ms/step - accuracy: 0.9969 - loss: 0.0122 - val_accuracy: 0.9903 - val_loss: 0.0396

```

🔴 CELL 8 – Fusion (PRE + POST Average)

```
# =====
# CELL 8: Fusion Models
# =====

def fusion_model(model_type, name):
    model_pre = build_model(model_type)
    model_post = build_model(model_type)

    model_pre.fit(X_train_pre, y_train, epochs=10, batch_size=128, verbose=0)
    model_post.fit(X_train_post, y_train, epochs=10, batch_size=128, verbose=0)

    pred_pre = model_pre.predict(X_test_pre)
    pred_post = model_post.predict(X_test_post)

    preds = ((pred_pre + pred_post) / 2 > 0.5).astype("int32")

    acc = accuracy_score(y_test, preds)
    prec = precision_score(y_test, preds)
    rec = recall_score(y_test, preds)
    f1 = f1_score(y_test, preds)

    return [name, acc, prec, rec, f1]

results.append(fusion_model('LSTM', "LSTM_PRE+POST_FUSION"))
results.append(fusion_model('GRU', "GRU_PRE+POST_FUSION"))
results.append(fusion_model('RNN', "RNN_PRE+POST_FUSION"))
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:97: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
281/281 ————— 2s 5ms/step
281/281 ————— 2s 5ms/step
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:97: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
281/281 ————— 2s 5ms/step
281/281 ————— 2s 5ms/step
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:97: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
281/281 ————— 3s 7ms/step
281/281 ————— 2s 7ms/step
```

🔥 CELL 9 – Final Results Table (LIKE YOUR FORMAT)

```
# =====
# CELL 9: Final Results Table
# =====

results_df = pd.DataFrame(results, columns=["Model", "Accuracy", "Precision", "Recall", "F1-Score"])

results_df
```

	Model	Accuracy	Precision	Recall	F1-Score
0	LSTM_PRE	0.985078	0.983403	0.985246	0.984324
1	LSTM_POST	0.984410	0.976026	0.991569	0.983736
2	GRU_PRE	0.988864	0.982639	0.994145	0.988359
3	GRU_POST	0.987973	0.988727	0.985948	0.987336
4	RNN_PRE	0.963808	0.967306	0.956206	0.961724
5	RNN_POST	0.799777	0.708081	0.985012	0.823898
6	LSTM_PRE+POST_FUSION	0.986414	0.984806	0.986651	0.985728
7	GRU_PRE+POST_FUSION	0.987639	0.984168	0.989930	0.987040
8	RNN_PRE+POST_FUSION	0.975724	0.970725	0.978454	0.974574
9	LSTM_PRE+POST_FUSION	0.987082	0.982574	0.990398	0.986471
10	GRU_PRE+POST_FUSION	0.990089	0.991074	0.988056	0.989563
11	RNN_PRE+POST_FUSION	0.971158	0.968028	0.971429	0.969725

Start coding or [generate](#) with AI.